

Progree DevOps Internship

Task 3: Multi-Stage Automated CI/CD Deployment Pipeline

Intern: Haseeb Ullah

Application: Wanderlust frontend, backend, MongoDB, and Redis

Platform: GitHub Actions, Ubuntu runners, Docker Compose

The objective is to automate testing and integration on remote code pushes and show deployment results on an execution board. This project checks application quality, builds the images, deploys the real stack, verifies it, and records the outcome without manual deployment commands.

Requirement coverage

Internship requirement	Delivered implementation
Run on remote pushes	GitHub push trigger; actual successful and deliberately failed runs recorded.
Automatically pull code	Pinned checkout action in every job; each stage uses the triggering revision.
Static application linting	ESLint for backend/frontend; separate frontend TypeScript check.
Unit-testing suites	28 backend unit tests and 12 frontend unit/component tests.
Multi-stage deployment	Quality checks gate image builds. A separate job loads the built image archive and deploys the four-service application.
Deployment status metrics	Actions job graph, summaries, test counts/coverage, startup duration, commit identity, health, logs, and saved artifacts.

Execution flow

Remote push

- > Lint + type checks + unit/component tests
- > Build images tagged with the commit SHA
- > Transfer image archive + checksum + manifest
- > Deploy Compose stack on a fresh runner
- > Verify image IDs, commit, readiness, and API
- > Save results and remove test environment

Deployment scope: a temporary GitHub-hosted test environment, not a persistent public website. The application runs during verification and is removed afterward. The Ubuntu Task 2 demo is separate.

Application attribution: Wanderlust by Krishna R Acharya and contributors, MIT license. The portfolio contribution is the DevOps configuration, verification, and documentation.

Successful deployment evidence

Successful push-triggered pipeline: [GitHub run 36829758268](#)

Commit: dd36bc884d8e66c7e803f96a4be21cd5972f14c7

Recorded: 2026-10-01T07:20:29Z

Conclusion: success

Job	Outcome
Lint and unit tests (backend)	success
Lint and unit tests (frontend)	success
Build immutable application images	success
Deploy and verify the test environment	success
Pipeline status board	success

Measured results

Metric	Recorded result
Unit/component tests	28 backend + 12 frontend passed; 0 failed
Deployment startup + revision verification	30.16 seconds
Backend / MongoDB / Redis	All ready
Live application verification	success
Environment cleanup	success

Checks against the running application

PASS: Frontend and direct SPA navigation respond

PASS: Backend, MongoDB, and Redis are ready through the Nginx proxy

PASS: Invalid post is rejected

PASS: Email/password API signup and signin use the runtime JWT secret

PASS: Create, read, and list a post backed by MongoDB

PASS: Update is visible in both detail and cached list routes

PASS: Delete removes the post from MongoDB and cached lists

The image checksum is verified before loading. Running container image IDs and revision labels must match the build manifest. The API health endpoint must report the same commit SHA as the push. Deployment uses --no-build, so the tested images are reused.

Failure gate and recovery

[Controlled failed-test demonstration: GitHub run 36830344712](#)

Commit: d1ee9f1216608ef06913ec9aedb407b089fafa8b

Demonstration: a test deliberately expected an invalid category to be accepted. The backend test failed while linting and the frontend checks passed.

Job	Outcome
Lint and unit tests (backend)	failure
Lint and unit tests (frontend)	success
Pipeline status board	success
Build immutable application images	skipped
Deploy and verify the test environment	skipped

The failed branch was eligible for deployment under the same push trigger. Build and deployment were skipped because the quality job failed. The reporting job still ran, while the overall workflow remained failed. The deliberately failing test was reverted and was never merged into main.

[Successful rerun after reverting the deliberate failure: GitHub run 36830587646](#)

Why the checks can be trusted

The inherited backend teardown called `process.exit(0)`; it was removed so Jest returns a failure status for failed tests. Frontend API fixtures make component tests independent of a preloaded local database. Lint and type-check errors also fail the workflow. No quality stage uses `continue-on-error`.

Coverage reporting

Selected-module coverage	Backend	Frontend
Lines	87.17%	93.75%
Branches	75%	92.85%
Functions	92.3%	85%
Statements	86.41%	93.75%

Coverage applies only to the selected modules declared in the Jest configurations. It is reported as a metric, not presented as whole-application coverage. Real database/cache behavior is also checked after deployment.

Operation and submission handoff

Where to review the pipeline

[Open the Task 3 GitHub Actions workflow](#)

Choose a run to see job outcomes, test summaries, deployment metrics, logs, and artifacts.

Artifact	Contents / retention
tests-backend-* / tests-frontend-*	Test JSON and selected-module coverage; 14 days.
build-output-*	Docker build logs; 14 days.
application-images-*	Image archive, checksum, and manifest; 3 days.
deployment-evidence-*	Commit, startup time, health, API checks, service status, and redacted logs; 14 days.

The repository also retains compact evidence under docs/task-3/evidence so the report remains useful after downloadable artifacts expire. GitHub run links provide the original execution history.

Important operational behavior

Pushes and manual runs deploy temporary test environments. Pull requests run quality and build checks without deployment. Each deployment has its own Compose project name and temporary credentials. Runner credentials are masked in logs; uploaded paths exclude secret files. The GitHub token has read-only repository-content permission, and action versions are pinned to commit hashes.

Run the same quality checks locally

```
npm ci --prefix backend
npm ci --prefix frontend
npm run lint --prefix backend
npm run lint --prefix frontend
npm run typecheck --prefix frontend
npm run test:ci --prefix backend
npm run test:ci --prefix frontend
```

Submission status

Task 3 satisfies the stated automation objective based on the linked successful deployment and failed-test gate demonstration. This report is a component for the final combined internship PDF. It has not been submitted to the Google Form. The stated deadline is 5 October 2026. Task 4 remains separate infrastructure/Kubernetes work.

References

[GitHub workflow syntax](#)

[GitHub workflow commands and job summaries](#)